OPERATIONS WORKBOOK

Evidence Comparison and Correction Workbook

A three-attempt review system for isolating one persistent gap, choosing a correction, and running a comparable re-test.

Find the repeated decision moment behind the score, change one behavior, and require evidence of transfer.

VERSION 1.1

FORMAT GUIDE + WORKED EXAMPLES

STATUS LEARNER READY

OPERATIONS / EVIDENCE-COMPARISON-CORRECTION

COMPARE WITH DISCIPLINE

Find the repeated decision, not the lowest number

Three attempts become useful only when the prompt shape, conditions, and rubric are comparable enough to reveal a recurring behavior.

CORE IDEA

A persistent gap is a repeated observable moment under comparable conditions. One awkward answer is a hypothesis; three low totals without evidence are still only three numbers.

01	02	03	04
Normalize	**Quote**	**Cluster**	**Correct**
Use the same dimensions and record prompt difficulty and conditions.	Capture the exact moment the decision became unclear or late.	Look for repeated cues, causes, and downstream consequences.	Choose one front-loaded behavior and define the re-test.

PERSISTENT	ONE-OFF
Signal	**Noise**
The same behavior appears in similar decision moments and affects correctness, clarity, trade-offs, or recovery.	The prompt, time, assistance, fatigue, rubric, or domain changed enough that the result is not directly comparable.

Before comparing

- ☐ Prompt type and difficulty are recorded.
- ☐ Time, tools, and assistance are visible.
- ☐ The same rubric dimensions were used.
- ☐ Evidence includes quotes, timestamps, code/tests, diagrams, or reviewer notes.

USE ONE SHARED RUBRIC

Compare behavior across interview types

The wording changes by domain, but the comparison should still expose framing, decision quality, boundary handling, and communication.

Dimension	Coding evidence	System-design evidence	Behavioral evidence
Framing	Contract, examples, constraints	Requirements, quality attributes, non-goals	Question, context, stakes
Decision	Pattern and state choice	Architecture/data choice	Your choice, not only team action
Reason	Invariant and complexity proof	Requirement and trade-off	Constraint, alternative, judgment
Boundary	Empty, duplicate, off-by-one	Failure, overload, stale state	Conflict, uncertainty, follow-up
Evidence	Dry run, tests, correct output	Estimate, metric, recovery behavior	Measured result and durable change
Communication	Decisions audible at the right time	Flow remains traceable as design expands	Scope is clear without excessive setup

SCORING DISCIPLINE

Use the same 0 / 1 / 2 evidence scale from the Technical Baseline Scorecard, but compare the notes behind each score. The number is an index into evidence, not the conclusion.

A useful quote

Record enough surrounding context to understand the decision. 'I forgot consistency' is weaker evidence than: 'I selected asynchronous replication, then described read-your-writes only after the interviewer asked what a user sees after an update.'

FIND THE REPEATED MOMENT

Trace symptom, cause, and downstream consequence

Do not stop at the visible mistake. Find the earliest decision whose absence made later parts of the answer harder to recover.

Repeated symptom	Possible earlier cause	Downstream consequence	Correction candidate
Edge cases appear only after prompting	The input contract and invariant were never stated	Implementation changes late; testing becomes random	Name contract and one boundary before coding
Design becomes a list of components	Requirements and first bottleneck were not used as decision anchors	Trade-offs sound generic; failures are bolted on	State requirement -> decision -> cost for each major box
Ownership sounds team-wide	The speaker never named the personal decision boundary	Level and judgment are hard to calibrate	Open with context, then 'My decision was...'
Follow-up answers contradict the opening	Assumptions were implicit and changed silently	The interviewer cannot tell whether the design adapted coherently	Label the assumption before changing the decision

Root-cause questions

- What cue was present before the gap appeared?
- Which decision should have been made at that moment?
- Did the same absence create problems in more than one attempt?
- Would correcting it improve several downstream dimensions?
- Can the behavior be observed in the next comparable attempt?

CHOOSE THE SMALLEST CORRECTION

Prefer the earliest observable behavior with broad downstream impact. Do not replace a correction with a new syllabus.

WORKED EXAMPLE

From three attempts to one correction

The fictional evidence below shows how an apparent knowledge gap can actually be a decision-order problem.

Attempt	Prompt	Observed moment	Consequence
A	Notification system	Retry policy added after the queue diagram	Duplicate effects and overload path stayed vague
B	File processing	Consumer failure discussed only after a follow-up	No dead-letter or poison-message plan
C	Metrics pipeline	Delivery semantics named after storage choice	Reconciliation and idempotency contradicted earlier claims

TOO BROAD

Weak conclusion

I need to learn queues again. This is broad, hides what was already known, and gives no moment to test.

SPECIFIC

Evidence-led conclusion

I understand queue components, but delivery and retry semantics appear after architecture. Front-load them immediately after requirements.

CORRECTION

Before drawing components, state: critical work is at-least-once; consumers are idempotent; retries are bounded with dead letters; oldest-event age is the recovery signal.

Re-test condition

Use a nearby asynchronous-workflow prompt, the same 35-minute structure, and the same rubric. Success means delivery/retry behavior appears before architecture and remains coherent during one failure follow-up.

WORKED COMPARISON

Compare three attempts without hiding the evidence

These fictional runs use comparable asynchronous-workflow prompts and the same six dimensions. Read each score with its timestamp; the repeated sequence matters more than the totals.

FICTIONAL WORKED EXAMPLE - ADAPT THE METHOD, NOT THE FACTS

Context	Attempt A	Attempt B	Attempt C
Date / prompt	Example Day 1: notifications	Example Day 3: file processing	Example Day 5: metrics pipeline
Type / difficulty	Async system design; moderate scope	Async system design; moderate scope	Async system design; moderate scope
Time / tools / assistance	35 min; blank canvas; no hints	35 min; blank canvas; no hints	35 min; blank canvas; no hints
Rubric used	Shared six dimensions; 0/1/2 plus artifacts	Same dimensions and scoring rule	Same dimensions and scoring rule

FICTIONAL WORKED EXAMPLE - ADAPT THE METHOD, NOT THE FACTS

Shared dimension	Attempt A: 0/1/2 + evidence	Attempt B: 0/1/2 + evidence	Attempt C: 0/1/2 + evidence
Framing	2: 02:10 identifies accepted versus delivered.	2: 02:40 defines upload and processing result.	2: 03:00 separates ingestion from query freshness.
Decision	1: 08:00 queue drawn before delivery guarantee.	1: 09:20 workers drawn before crash behavior.	1: 08:40 store chosen before delivery semantics.
Reason	1: retry cost named only after queue choice.	1: parallelism chosen without poison-message boundary.	1: throughput discussed; replay cost omitted.
Boundary	0: crash after email acceptance unresolved.	1: consumer failure addressed at 27:00 after probe.	1: duplicates addressed at 28:30 after probe.
Evidence	1: diagram exists; no duplicate timeline.	1: retry sketch exists; no dead-letter re-drive test.	1: retention estimate exists; reconciliation untested.
Communication	2: major choices audible; assumptions dated.	2: scope changes explicitly named.	2: flow remains traceable despite late semantics.

WORKED SYNTHESIS

Turn score notes into a defensible pattern

The synthesis below uses the same three attempts. It preserves strengths, separates domain differences from the repeated gap, and states what is still uncertain.

FICTIONAL WORKED EXAMPLE - ADAPT THE METHOD, NOT THE FACTS

Synthesis	Evidence across Attempts A / B / C
Strongest repeated behavior	All three answers clarify user-facing goals early and keep major choices audible. Preserve those habits; the correction is not "communicate more."
First repeated weak decision moment	At about minutes 8-9, the learner draws storage or workers before stating delivery, retry, and replay guarantees.
Cue present immediately before it	The prompt introduces asynchronous work that can be retried after a crash. That cue should trigger a semantics checkpoint.
Quote / timestamp / artifact references	A 08:00 queue sketch; B 09:20 worker sketch and 27:00 failure probe; C 08:40 storage choice and 28:30 duplicate probe.
What is noise or still uncertain?	Domains differ, so totals are not directly comparable. The repeated ordering gap is stronger evidence than the exact score change; external-provider guarantees still need verification.

COMPARISON RULE

A score without an evidence note is incomplete. If prompt type, difficulty, assistance, or time changed enough to explain the difference, mark the comparison as uncertain instead of forcing a trend.

WORKED CORRECTION

Turn the pattern into one observable behavior

Follow the chain from the repeated architecture-first moment to a specific checkpoint. In your notebook, repeat this chain using your own quotes and artifacts.

FICTIONAL WORKED EXAMPLE - ADAPT THE METHOD, NOT THE FACTS

Step	Illustrative evidence
Repeated observable moment	Architecture appears before failure and retry semantics in all three asynchronous-workflow attempts.
Cue present before it	Accepted work can outlive the request and be retried after a worker or dependency failure.
Earliest missing decision	Define the durable acceptance boundary, duplicate-effect rule, bounded retry policy, and recovery owner.
Downstream consequence	Failure follow-ups force late changes to IDs, state transitions, metrics, and reconciliation.
One correction sentence	When work becomes asynchronous, I will state delivery and duplicate-effect guarantees before drawing workers so retries and recovery follow the same contract.
Evidence that would show improvement	Checkpoint appears unaided before architecture; crash-after-side-effect probe is answered without an unexplained guarantee change.

CORRECTION SENTENCE TEMPLATE

When I notice [cue], I will [observable behavior] before [decision point], so [quality or failure consequence] stays explicit.

Keep / change

COMPARABLE

Keep stable

Prompt shape, time box, tools, assistance, rubric dimensions, and scoring evidence.

ATTRIBUTION

Change once

The one intended behavior. Do not add several new techniques during the same re-test.

WORKED RE-TEST

Repeat the weak decision window, then test transfer

This fictional 35-minute block contains 28 minutes on the original weak scenario and a seven-minute adjacent probe. Its evidence is not a new full-length baseline score.

FICTIONAL WORKED EXAMPLE - ADAPT THE METHOD, NOT THE FACTS

Re-test setup	Illustrative plan
Phase 1: original lowest-score scenario	Repeat notification delivery, the scenario with Boundary = 0; replay the original requirements-to-failure decision window.
Same prompt conditions / decision-window time box / tools	28 minutes; same prompt scope, blank canvas, no hints, and six-dimension evidence rubric. This is a focused replay, not a shorter full baseline comparison.
Correction cue	A notification is accepted now but may be delivered later and retried.
Where the behavior must appear	Immediately after requirements, before API persistence or queue-worker boxes.
Primary success evidence	Record the checkpoint timestamp and a crash-after-provider-acceptance timeline; do not assume the provider prevents duplicates.
Phase 2: 7-minute adjacent transfer probe	Switch to file processing: worker crashes after writing output but before acknowledging work. Explain stable job ID, output state, and safe retry.

FICTIONAL WORKED EXAMPLE - ADAPT THE METHOD, NOT THE FACTS

Result	Evidence / quote / timestamp	Next decision
Phase 1: same scenario	04:10: acceptance is durable before acknowledgment; duplicate email effects need provider idempotency or reconciliation. At 18:30, failure probe reuses this boundary.	Keep correction; confirm the provider capability rather than claiming exactly-once delivery.
Phase 2: brief transfer probe	02:00: output path uses stable job ID; retry checks durable completion before writing again. A stale partial object is quarantined.	Transfer observed once; return to roadmap and check again during the next normal rehearsal.

STOP CONDITION

Return to the main roadmap when the correction appears unaided in the comparable re-test and at least once in a nearby prompt. Do not rehearse until the answer becomes a script.

FINISH THE REVIEW

Make the next test smaller and clearer

The review is complete when one repeated gap has evidence, one correction has a visible moment, and the re-test conditions are recorded.

01	02	03	04
Compare	**Quote**	**Correct**	**Re-test**
Normalize conditions and use one rubric.	Capture repeated moments, not vague impressions.	Choose the earliest behavior with downstream impact.	Change one variable and check transfer.

Common mistakes

+ Picking the lowest score without reading the evidence behind it.
+ Comparing prompts or conditions that are not meaningfully comparable.
+ Calling a single bad moment persistent because it felt uncomfortable.
+ Choosing several corrections and losing attribution.
+ Repeating the identical answer until it becomes memorized rather than transferable.

YOUR NEXT ACTION

Complete the comparison grid, write one correction sentence, and schedule one comparable re-test plus one adjacent transfer check.

Completion check

- [ ] Three attempts use the same dimensions and include evidence.
- [ ] A repeated decision moment is separated from noise.
- [ ] One correction is attached to a cue and decision point.
- [ ] The comparable re-test and transfer check are defined.

APPLY THE METHOD TO YOUR OWN EVIDENCE

Use your last three artifacts: normalize conditions, attach evidence to the same six dimensions, write one repeated cue-to-consequence chain, and schedule the 28-minute replay plus seven-minute transfer probe.